

Make — The Task Runner

Why a decades-old build tool survives as the project's command shortcuts — `make tests`, `make check` — and what a Makefile target really is.

SUBJECT hathor-core · Part I · Track C (the stack)

CHAPTER 14 · Foundations · The Stack

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Make · Makefile · Targets · Recipes · .PHONY · Variables · Task runner vs build system · Self-documenting commands

Table of Contents

Chapter 14 — Make: The Task Runner	3
14.1 What Make was built for	3
14.2 Anatomy of a Makefile	4
14.3 .PHONY: targets that aren't files	4
14.4 What the real Makefile does	5
14.5 Why Make, and not the alternatives?	5
14.6 Bridge — Make across the project	6
Recap	6

Chapter 14 — Make: The Task Runner

What you'll learn in this chapter

- What **Make** originally was (a build system for C) and how projects repurpose it as a **task runner**.
- The anatomy of a **Makefile**: targets, recipes, prerequisites, and variables.
- What **.PHONY** means and why almost every target in `hathor-core`'s Makefile is phony.
- How the real Makefile bundles long tool invocations into short commands (`make tests`, `make check`, `make docker`).
- **Why Make** for this and not a pile of shell scripts or just typing the commands — the trade-off.

You will see instructions like `make tests` and `make check` throughout a project, and the file behind them — the `Makefile` — can look cryptic. **Make**¹ is one of the oldest tools in software (from 1976), built for a different purpose than the one it usually serves today. Understanding both the original purpose and the modern repurposing makes the file legible. `hathor-core` uses Make as a **task runner**²: a single, discoverable place that names the project's common commands so nobody has to memorize long invocations.

This is a short \$5-style primer; Make is a means, not a deep subsystem, so the chapter is proportionate.

14.1 What Make was built for

Make's original job was **building C programs**, and its core idea is genuinely clever for that. A C project compiles many source files into object files, then links them into a program. Recompiling everything on every change is wasteful, so Make tracks **dependencies** between files and their **timestamps**: it rebuilds a target only if something it depends on changed more recently than the target itself. "Rebuild `program` from `a.o` and `b.o`; rebuild `a.o` from `a.c` only if `a.c` is newer." This dependency-and-timestamp engine is what makes Make a **build system**³.

That machinery earns its keep for compiled languages. Python, though, is not compiled by you (Chapter 11 — CPython compiles bytecode automatically), so a Python project rarely needs Make's timestamp-driven rebuild logic. What it does want is the other half

of Make: a tidy way to define named commands. So Python projects use Make as a **task runner** — keeping the "named commands" feature and ignoring the "rebuild-if-newer" feature. Knowing this split is the key to reading the Makefile without confusion.

14.2 Anatomy of a Makefile

A Makefile is a list of **rules**. Each rule has this shape:

```
target: prerequisites
    recipe          ← a shell command (MUST be indented with a TAB, not spaces)
```

- **Target** — usually a filename to build, but for a task runner, just a name you invoke (`tests` , `check`).
- **Prerequisites** — other targets/files that must be ready first (often empty for task-runner targets).
- **Recipe** — the shell command(s) to run, each line indented by a literal **tab** (a famous Make gotcha: spaces won't work).

You invoke a rule by name: `make tests` runs the `tests` target's recipe. Make also supports **variables** to avoid repetition. The real Makefile opens with one (`Makefile:1`):

```
py_sources = hathor/ hathor_tests/ extras/custom_tests/
```

`$(py_sources)` then expands to that list wherever it's used, so the set of source directories is defined once and reused across the lint, format, and type-check targets — change it in one place, every target follows.

14.3 `.PHONY`: targets that aren't files

Here is the one concept that trips up newcomers reading a task-runner Makefile. Recall Make's origin: a target is normally a file to build, and Make decides whether to run the recipe by checking if that file is out of date. But `tests` is not a file — there is no file named `tests` in the project, and we want `make tests` to run every time, not be skipped because "the `tests` file is up to date."

`.PHONY`⁴ declares a target as not a real file, so Make always runs its recipe regardless of timestamps or any same-named file. In a task-runner Makefile, essentially every target is phony, and `hathor-core`'s Makefile marks them as such (the `.PHONY:` lines throughout). When you see `.PHONY: tests`, read it as "`tests` is a command, not a file — always run it." This single declaration is what converts Make from a build system into a reliable task runner.

14.4 What the real Makefile does

`hathor-core`'s Makefile is a catalogue of the project's routine commands, each wrapping a longer tool invocation. The clusters:

Testing. Several targets run `pytest` with different scopes and coverage settings — `tests-cli`, `tests-lib`, `tests-nano`, `tests-genesis`, `tests-quick` — and an aggregate `tests` (Makefile:63) that runs the suite. For example, `tests-cli` (:33) runs `pytest` scoped to `hathor_cli/` with a coverage floor. The point: a contributor types `make tests` instead of remembering a 100-character `pytest` command with the right flags.

Quality checks. `mypy` (:72) runs the type checker over the packages; `flake8` (:80) lints `$(py_sources)`; `isort-check` (:84) verifies import ordering; `yamllint` checks YAML. The aggregate `check` (:96) runs all of them together — this is the single command a contributor (or CI) runs to validate a change before submitting. One name, the whole quality gate (Chapter 20).

Formatting. `isort` (:107) sorts imports in place; `fmt` (:104) is a friendly alias for it. The difference between `isort` and `isort-check` is fix vs. verify: `make fmt` edits your files, `make check` only reports.

Docker. `docker` (:144) builds the image; related targets push it to registries (Chapter 15). The tag is derived from the git tag or a timestamp — logic you'd never want to retype.

The Makefile, read top to bottom, is therefore a **self-documenting list of how to work on the project**: how to test it, check it, format it, and ship it. That is its real value — not the build engine, but the index of commands.

14.5 Why Make, and not the alternatives?

vs. just typing the commands. You could type the full `pytest --cov=... --cov-fail-under=... -p no:warnings ...` each time. Nobody remembers it correctly, everyone runs it slightly differently, and CI drifts from local. `make tests` makes the canonical command the only command — one source of truth that local development and CI share.

vs. a folder of shell scripts. Shell scripts (`scripts/test.sh`, `scripts/check.sh`) would also centralize commands, and many projects use them. Make's edges: everything lives in one discoverable file (`make` with no clean listing, or a glance at the Makefile, shows all available tasks), variables like `$(py_sources)` are shared across tasks without sourcing, and `make` is preinstalled almost everywhere a developer or CI runner exists. The cost is Make's awkward syntax (tabs-not-spaces, its own variable and escaping rules) and that its real power — the dependency graph — goes unused.

vs. a dedicated Python task runner (invoke, nox, tox, just). These are purpose-built for the task-runner job and avoid Make's archaic syntax. They're reasonable choices; `just` in particular is a modern Make-for-tasks. `hathor-core` stays on Make for the usual reason: it is universal, zero-install, and already works. The team would gain cleaner syntax and lose universality and a working setup — rarely worth it for a tool this peripheral.

The honest summary: Make is a 1970s build system pressed into service as a task runner, kept because it is everywhere and already does the job. You accept its quirky syntax and ignore its build-graph engine; you get a single, shared, zero-install index of every command needed to work on the project.

14.6 Bridge — Make across the project

Bridge — Make in the project and the stack:

- **It wraps the quality toolchain.** `make check` runs flake8 + isort-check + mypy + yamllint together — the Chapter 20 gate behind one command — **Chapter 20**.
- **It wraps Poetry/pytest invocations.** The test targets call pytest with the right coverage flags; contributors don't retype them — **Chapters 13 & 20**.
- **It builds the Docker image.** `make docker` runs the build with the right tag logic — **Chapter 15**.
- **It is the contributor's entry point.** The Makefile is the discoverable list of "how to develop this project," sitting above the individual tools.

Recap

Concept	What it is	In the Makefile
Make	build tool repurposed as task runner	the <code>Makefile</code>
target	a named command (here) or file (originally)	<code>tests:</code> , <code>check:</code> , <code>docker:</code>
recipe	shell command(s) under a target, tab-indented	the <code>pytest/mypy/flake8</code> lines
variable	a reused value	<code>py_sources (:1)</code>
<code>.PHONY</code>	"not a file — always run"	nearly every target
<code>make check</code>	run the whole quality gate	<code>:96</code>

Concept	What it is	In the Makefile
<code>make fmt</code> vs <code>check</code>	fix in place vs. verify only	<code>:104</code> vs <code>:84</code>
why Make	universal, zero-install, one file	vs scripts / just / typing it

Make is an old build system that survives in modern Python projects as a task runner: a single, discoverable file that names every routine command — test, lint, type-check, format, build the image — and wraps each long tool invocation behind a short, shared name. The `.PHONY` declarations are what repurpose it, telling Make these targets are commands to always run, not files to conditionally rebuild. It is kept over shell scripts and newer task runners for being universal and already working, at the cost of archaic syntax. The next chapter takes the isolation theme one level deeper than the virtual environment: **Docker**, which packages not just Python dependencies but the entire operating environment the node runs in.

-
1. Make is a build-automation tool (originally 1976) that runs commands defined in a `Makefile`. Built to compile programs by rebuilding only out-of-date files, it is widely repurposed as a task runner for named commands. ↩
 2. A task runner is a tool that defines and runs a project's common named commands (test, lint, build) from one place, so contributors and CI invoke identical commands without memorizing them. ↩
 3. A build system turns source into artifacts, tracking dependencies and timestamps to rebuild only what changed. Make is one; for compiled languages this incremental rebuild is its main value, mostly unused in Python projects. ↩
 4. A phony target (declared with `.PHONY`) is a Make target that does not correspond to a file, so Make runs its recipe every time rather than skipping it based on file timestamps. Essential when using Make as a task runner. ↩